MODULE 12

Unsupervised Learning + Model Evaluation & Feature Engineering

Detailed Student Study Material

Concept Explanations • Worked Examples • Exam Preparation • Expected Practicals with Answers

Module Coverage
Unsupervised Learning fundamentals, clustering, K-Means, hierarchical clustering, dimensionality reduction, PCA, feature engineering, classification evaluation metrics, cross-validation, hyperparameter optimization, analytical storytelling, AI-assisted analysis, and hands-on business analytics tasks.

Prepared as a classroom and exam revision guide

Contents

1. 1. Learning Outcomes
2. 2. Unsupervised Learning Fundamentals
3. 3. Clustering Algorithms
4. 4. Dimensionality Reduction and PCA
5. 5. Feature Engineering
6. 6. Model Evaluation
7. 7. Cross Validation
8. 8. Hyperparameter Optimization
9. 9. Analytical Storytelling
10. 10. Thinking with AI
11. 11. End-to-End Workflow
12. 12. Expected Theory Questions
13. 13. Dummy Dataset for Practicals
14. 14. Expected Practical Questions with Complete Answers
15. 15. Additional Practice Tasks
16. 16. Quick Revision Sheet

1. Learning Outcomes

After completing this module, a student should be able to explain and apply the following skills:

- Explain unsupervised learning and distinguish it from supervised learning.
- Identify patterns in unlabeled data and perform customer segmentation.
- Build and interpret K-Means and hierarchical clustering models.
- Understand high-dimensional data and reduce dimensionality using PCA.
- Create, transform, select, and encode useful features.
- Compute and interpret Accuracy, Precision, Recall, F1 Score, and a Confusion Matrix.
- Use K-Fold Cross Validation to estimate model generalization.
- Explain parameters versus hyperparameters and tune models using Grid Search and Random Search.
- Convert technical outputs into business recommendations and communicate uncertainty.
- Use AI tools for suggestions while validating code, metrics, and conclusions.

Important Note
The module combines unsupervised learning topics with general model-evaluation topics. Accuracy, Precision, Recall, F1 Score, and the Confusion Matrix are mainly classification metrics, while clustering needs different evaluation approaches such as inertia or silhouette score when labels are not available.

2. Unsupervised Learning Fundamentals

2.1 What Is Unsupervised Learning?

Unsupervised Learning is a machine learning approach in which the algorithm receives input data without a predefined target label. The objective is not to predict a known answer, but to discover hidden structure, patterns, similarities, groups, or compact representations inside the data.

Supervised Learning	Unsupervised Learning
Uses labeled data	Uses unlabeled data
Learns input-to-target mapping	Discovers hidden structure
Common tasks: classification, regression	Common tasks: clustering, dimensionality reduction
Evaluation is usually direct	Evaluation can be less direct
Example: predict churn	Example: segment customers

2.2 Learning from Unlabeled Data

Unlabeled data contains features but no correct output column. For example, a retail company may know each customer's age, annual income, spending score, number of visits, and average bill amount, but may not know whether the customer belongs to a premium, regular, or low-engagement segment. Clustering algorithms can discover such groups automatically.

Customer	Age	Income	Spending Score
C1	23	28,000	81
C2	25	31,000	76
C3	45	90,000	24
C4	48	96,000	20

2.3 Pattern Discovery

Pattern discovery means identifying recurring or hidden relationships in data. A pattern may represent similar buying behavior, similar usage behavior, unusual transactions, or natural groupings of observations. These patterns are often not visible from a single row; they become clear only after analyzing many rows together.

- Frequent buyers who purchase every month
- High-income customers with low spending
- Customers who mainly purchase during offers
- Transactions that differ strongly from normal behavior
- Products that are repeatedly purchased together

2.4 Segmentation Concepts

Segmentation divides a large population into smaller groups whose members share similar characteristics. Customer segmentation is one of the most common business uses of clustering. It allows businesses to use different strategies for different groups instead of treating every customer in the same way.

Possible Segment	Typical Characteristics	Possible Action
Premium	High spending, frequent visits	Loyalty rewards
Regular	Medium spending, stable visits	Cross-sell offers
Price-sensitive	Low spending, high discount usage	Target promotions
At-risk	Previously active, now declining	Re-engagement campaign

2.5 Business Applications

- Customer segmentation and targeted marketing
- Fraud and anomaly pattern discovery
- Grouping similar documents or news articles
- Product grouping and catalog organization
- Image segmentation
- Behavioral analysis in websites and apps
- Preprocessing before downstream modeling

3. Clustering Algorithms

3.1 What Is Clustering?

Clustering is the process of grouping observations so that points in the same cluster are more similar to one another than to points in other clusters. Similarity is often measured using distance, such as Euclidean distance, although the exact method depends on the algorithm.

3.2 K-Means Clustering

K-Means is a centroid-based clustering algorithm. The letter K represents the number of clusters. Each cluster is represented by a centroid, which is the mean position of the points assigned to that cluster.

17. Choose the number of clusters K.
18. Initialize K centroids.
19. Calculate the distance from each observation to each centroid.
20. Assign each observation to the nearest centroid.
21. Recalculate each centroid as the mean of its assigned points.
22. Repeat assignment and centroid calculation until the clusters stabilize.

Why Scaling Matters

K-Means uses distance. If one feature is measured in lakhs and another feature is measured from 1 to 10, the large-scale feature may dominate the distance calculation. StandardScaler is therefore commonly applied before K-Means.

3.3 K-Means Objective and Inertia

K-Means tries to minimize the sum of squared distances between each observation and its assigned centroid. In scikit-learn, this within-cluster sum of squares is available through the model's inertia_ attribute. Lower inertia means points are closer to their centroids, but inertia always decreases when more clusters are added. Therefore, it should not be used alone to choose K.

3.4 Choosing K with the Elbow Method

The Elbow Method fits K-Means for several values of K and plots K against inertia. The selected K is often near the point where additional clusters stop producing a large improvement. This bend in the graph resembles an elbow.

```
from sklearn.cluster import KMeans


inertias = []
for k in range(1, 8):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    inertias.append(model.inertia_)
```

3.5 Silhouette Score

Although not explicitly listed in the module, silhouette score is a standard practical method for evaluating cluster separation. It compares how close a point is to its own cluster versus neighboring clusters. Values closer to 1 indicate better-separated clusters, values around 0 indicate overlapping clusters, and negative values may indicate incorrect assignment.

3.6 Hierarchical Clustering

Hierarchical clustering builds a hierarchy of clusters rather than directly producing one flat grouping. The most common form is agglomerative clustering, which starts with every observation as its own cluster and repeatedly merges the closest clusters.

Approach	Direction	Process
Agglomerative	Bottom-up	Start with individual points and repeatedly merge
Divisive	Top-down	Start with one group and repeatedly split

3.7 Dendrogram and Linkage

A dendrogram is a tree diagram that records how clusters are merged. The height at which two branches join represents the distance between clusters. By drawing a horizontal cut through the dendrogram, the analyst can choose a useful number of clusters. Common linkage rules include single, complete, average, and Ward linkage.

3.8 Cluster Interpretation and Segment Analysis

A clustering algorithm usually returns numerical labels such as 0, 1, and 2. These labels have no business meaning by themselves. The analyst must summarize the average feature values for each cluster and assign meaningful descriptions such as "Premium Customers", "Budget Customers", or "Low Engagement Customers".

Cluster	Avg Income	Avg Spending	Avg Visits	Interpretation

0	31,000	78	8.2	Young / high engagement
1	94,000	27	2.4	High income / low engagement
2	66,000	61	5.7	Mid-high value regulars

4. Dimensionality Reduction and PCA

4.1 High-Dimensional Data

A dataset is high-dimensional when it contains many features. Image, text, genomics, sensor, and customer-behavior datasets can contain tens, hundreds, or thousands of features. High dimensionality can increase computation, introduce redundant information, make visualization difficult, and sometimes reduce the effectiveness of distance-based methods.

4.2 What Is PCA?

Principal Component Analysis (PCA) is a linear dimensionality-reduction technique. It transforms the original correlated variables into new uncorrelated variables called principal components. The first principal component captures the largest possible variance in the data; the second captures the next-largest amount while remaining orthogonal to the first, and so on.

- PCA creates new features rather than simply deleting old ones.
- Principal components are weighted combinations of the original variables.
- PCA is sensitive to scale, so standardization is commonly performed first.
- The explained variance ratio shows how much information each component retains.

4.3 Variance, Covariance, Eigenvectors, and Eigenvalues

Variance measures how widely a variable is spread. Covariance measures how two variables vary together. PCA identifies directions in feature space along which the data varies the most. Mathematically, these directions are eigenvectors of the covariance matrix, and the corresponding eigenvalues indicate how much variance each direction explains.

4.4 Feature Compression

If a dataset contains 30 correlated numerical features, PCA may be able to represent most of their useful variation using only a few principal components. For example, 10 components may retain 95% of the total variance. This reduces the number of inputs while preserving much of the information.

4.5 PCA for Visualization

Data with dozens of dimensions cannot be directly plotted. PCA can reduce it to two or three dimensions, making it possible to visualize clusters or structure using PC1 versus PC2. Such a plot is useful for exploratory analysis, although the new axes are combinations of original features and may not be directly interpretable.

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X_scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
print(pca.explained_variance_ratio_)
```

5. Feature Engineering

5.1 What Is Feature Engineering?

Feature Engineering is the process of creating, transforming, selecting, or encoding variables so that the data represents the problem more effectively for a machine learning model. Good features can improve model quality even without changing the algorithm.

5.2 Feature Creation

Raw Columns	Engineered Feature	Formula / Logic
quantity, unit_price	total_amount	quantity × unit_price
date_of_birth	age	current date − date of birth
joining_date	tenure_days	current date − joining date
total_spend, orders	average_order_value	total_spend / orders

5.3 Feature Transformation

Feature transformation changes the representation or scale of a feature. Common transformations include standardization, min-max normalization, logarithmic transformations for strongly skewed numerical variables, and binning continuous values into ranges.

- Standardization: z = (x - mean) / standard deviation
- Min-Max scaling: maps values to a fixed range such as 0 to 1
- Log transformation: reduces strong positive skew
- Date extraction: year, month, weekday, weekend flag

5.4 Feature Selection

Feature selection keeps useful original features and removes irrelevant or redundant ones. It can reduce training time and improve interpretability. Common approaches include filter methods, wrapper methods, and embedded methods.

Method Type	Examples	Idea
Filter	Correlation, chi-square, mutual information	Score features independently of the final model
Wrapper	Recursive Feature Elimination	Repeatedly test subsets using a model
Embedded	Lasso, tree feature importance	Selection occurs during model training

5.5 Handling Categorical Variables

Most machine learning algorithms require numerical input. Categorical variables must therefore be encoded. One-Hot Encoding is commonly used for nominal categories without an inherent order. Ordinal Encoding may be used when categories have a meaningful order.

```
import pandas as pd


df = pd.DataFrame({"city": ["Kochi", "Calicut", "Thrissur"]})
encoded = pd.get_dummies(df, columns=["city"], dtype=int)
print(encoded)
```

Exam Point
Label Encoding should not automatically be used for every text column. Assigning 0, 1, and 2 to nominal categories can create an artificial order that does not exist.

6. Model Evaluation

This section of the module covers classification evaluation metrics. These metrics compare predicted labels with known actual labels.

6.1 Confusion Matrix

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

6.2 Accuracy

Accuracy is the proportion of all predictions that are correct.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Accuracy works well when classes are reasonably balanced and the costs of errors are similar. It can be misleading for highly imbalanced datasets.

6.3 Precision

Precision answers: “Of all samples predicted as positive, how many were actually positive?”

Precision = TP / (TP + FP)

Precision is important when false positives are costly.

6.4 Recall

Recall answers: “Of all actual positive samples, how many did the model detect?”

Recall = TP / (TP + FN)

Recall is important when missing a positive case is costly, such as disease screening or fraud detection.

6.5 F1 Score

F1 Score is the harmonic mean of Precision and Recall. It is useful when both false positives and false negatives matter, especially with imbalanced data.

F1 = 2 × (Precision × Recall) / (Precision + Recall)

6.6 Worked Metric Example

Assume TP = 40, TN = 50, FP = 10, FN = 5.

Metric	**Calculation**	**Result**
Accuracy	(40+50)/(40+50+10+5)	0.857
Precision	40/(40+10)	0.800
Recall	40/(40+5)	0.889
F1	2×(0.8×0.889)/(0.8+0.889)	≈0.842

7. Cross Validation

7.1 Why Validation Matters

A single train-test split may accidentally produce an easy or difficult test set. Cross validation reduces dependence on one particular split and provides a more stable estimate of model performance.

7.2 K-Fold Cross Validation

In K-Fold Cross Validation, the training data is divided into K subsets called folds. The model is trained K times. Each time, one fold is used for validation while the remaining folds are used for training. The K validation scores are then averaged.

Fold Run	Training Folds	Validation Fold
1	2,3,4,5	1
2	1,3,4,5	2
3	1,2,4,5	3
4	1,2,3,5	4
5	1,2,3,4	5

7.3 Reliable Evaluation and Generalization

If cross-validation scores are similar across folds, the model is more consistent. A very large difference between folds may indicate instability, insufficient data, strong heterogeneity, or a problem with the validation design. Generalization means performing well on unseen data rather than merely memorizing training examples.

8. Hyperparameter Optimization

8.1 Parameters vs Hyperparameters

Parameters	Hyperparameters
Learned automatically from training data	Chosen before or during the training process
Examples: regression coefficients, neural network weights	Examples: max_depth, n_estimators, n_clusters
Part of the fitted model	Control model behavior and complexity

8.2 Grid Search

Grid Search evaluates every specified combination of hyperparameter values. It is systematic but can become expensive when many parameters or values are included.

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

params = {
    "max_depth": [2, 3, 5, None],
    "min_samples_split": [2, 5, 10]
}

grid = GridSearchCV(DecisionTreeClassifier(random_state=42), params, cv=5)
```

```
grid.fit(X_train, y_train)
print(grid.best_params_)
```

8.3 Random Search

Random Search samples a specified number of combinations from the search space. It is often more efficient than Grid Search when many hyperparameters are being tuned because it does not test every possible combination.

8.4 Improving Predictive Performance

23. Create a baseline model.
24. Evaluate using suitable metrics.
25. Improve data quality and features.
26. Use cross validation.
27. Tune hyperparameters.
28. Re-evaluate on an untouched test set.
29. Choose the model based on the business objective, not a single metric alone.

9. Analytical Storytelling

9.1 Translating Model Outputs

Analytical storytelling converts technical results into language that decision-makers can understand. Instead of reporting only “Cluster 2 centroid = [1.4, -0.7, 0.8]”, explain that the cluster represents frequent, high-spending customers with strong engagement.

9.2 Business Recommendations

A useful analysis connects findings to possible actions. For example, if a cluster contains frequent high-value customers, a loyalty program may be considered. Recommendations should be supported by the observed data and should not claim causation when the analysis only shows association.

9.3 Risk Interpretation and Uncertainty

- Explain that model outputs are estimates, not guarantees.
- Mention false positives and false negatives where relevant.
- State when cluster assignments are weak or overlapping.
- Mention limitations in sample size, data quality, or changing behavior.
- Separate evidence from assumptions and recommendations.

9.4 Decision Support

Machine learning supports decisions; it does not remove the need for business judgment. Cost, feasibility, ethics, policy, and operational constraints should be considered together with analytical results.

10. Thinking with AI

The module encourages students to use AI for feature-engineering suggestions, evaluation discussions, hyperparameter guidance, and analytical interpretation, while validating generated outputs and verifying conclusions.

AI Can Help With	Student Must Verify
Suggesting engineered features	Whether the feature causes data leakage
Explaining evaluation metrics	Whether the chosen metric fits the business problem
Suggesting tuning ranges	Whether the ranges are computationally sensible

Interpreting output	Whether the interpretation matches the actual data
Generating code	Whether the code runs and uses the correct columns

11. End-to-End Workflow

30. Define the business problem.
31. Collect and inspect the data.
32. Clean missing values and duplicates.
33. Perform exploratory analysis.
34. Create or transform useful features.
35. Scale numerical features when required.
36. Build clustering or predictive models.
37. Use PCA when dimensionality reduction is useful.
38. Evaluate models using appropriate metrics.
39. Use cross validation for reliable estimates.
40. Tune hyperparameters.
41. Interpret outputs and communicate business insights.
42. Document assumptions, limitations, and next steps.

12. Expected Theory Questions

1. What is unsupervised learning?
It is a machine learning approach that works with unlabeled data to discover hidden structures, groups, patterns, or compact representations without a known target variable.

2. What is the difference between supervised and unsupervised learning?
Supervised learning uses labeled data and predicts a known target; unsupervised learning has no target label and focuses on discovering structure such as clusters.

3. Explain K-Means clustering.
K-Means divides data into K clusters by repeatedly assigning points to the nearest centroid and recomputing centroids until the assignments stabilize.

4. Why is feature scaling important before K-Means?
Because K-Means is distance-based. Features with much larger numerical scales can dominate the Euclidean distance and distort the clusters.

5. What is a dendrogram?
A dendrogram is a tree diagram used in hierarchical clustering to show how observations or clusters are progressively merged and at what distances.

6. What is PCA?
PCA is a dimensionality-reduction method that transforms correlated original variables into new uncorrelated principal components ordered by explained variance.

7. What is feature engineering?
Feature engineering is the creation, transformation, selection, and encoding of variables to represent the learning problem more effectively.

8. Differentiate feature selection and PCA.
Feature selection keeps a subset of original variables; PCA creates new variables that are combinations of the original features.

9. Define Accuracy, Precision, Recall, and F1 Score.
Accuracy measures overall correctness; Precision measures reliability of positive predictions; Recall measures how many actual positives are detected; F1 balances Precision and Recall.

10. Why can accuracy be misleading?
In imbalanced data, a model may predict only the majority class and still obtain a high accuracy while failing to identify the minority class.

11. What is K-Fold Cross Validation?
The dataset is split into K folds. The model trains K times, each time validating on a different fold, and the scores are averaged.

12. What is generalization?
Generalization is a model's ability to perform well on unseen data rather than merely memorizing training data.

13. Differentiate parameters and hyperparameters.
Parameters are learned from data during fitting; hyperparameters are configuration values set before or around training that control model behavior.

14. Grid Search vs Random Search?
Grid Search tests all specified combinations; Random Search tests sampled combinations and is often more efficient for large search spaces.

15. What is analytical storytelling?
It is the process of translating technical model results into understandable findings, business implications, recommendations, risks, and uncertainty.

13. Dummy Dataset for Practicals

The following synthetic customer dataset is used in the practical exercises. It contains enough variation to demonstrate feature engineering, clustering, PCA, and evaluation workflows. The churn column is included only for the supervised model-evaluation exercises.

```
import pandas as pd


data = {
    "customer_id": [f"C{i:02d}" for i in range(1, 31)],
    "age":
[22,24,25,23,27,29,31,33,35,36,38,40,42,44,46,48,50,52,54,56,28,30,34,37,41,43,47,49,53,58],
    "annual_income":
[28000,30000,32000,29000,35000,38000,42000,45000,48000,51000,54000,58000,62000,68000,72000,7600
0,82000,88000,92000,98000,36000,40000,47000,53000,61000,66000,74000,80000,90000,105000],
    "spending_score":
[82,78,85,80,75,72,69,65,63,60,58,55,52,48,45,42,38,35,30,25,77,70,64,57,50,46,40,36,28,20],
    "visits_per_month": [10,9,11,10,8,8,7,7,6,6,5,5,5,4,4,4,3,3,2,2,9,8,7,6,5,4,4,3,2,1],
    "avg_order_value":
[1200,1350,1100,1250,1400,1500,1600,1700,1800,1900,2100,2200,2400,2600,2800,3000,3300,3500,3800
,4200,1450,1550,1750,2050,2350,2700,3100,3400,3900,4500],
```

```
    "membership_months":
[4,6,5,7,8,9,10,12,14,16,18,20,22,24,26,28,30,32,34,36,7,9,13,17,21,25,29,31,35,40],
    "city":
["Kochi","Calicut","Kochi","Thrissur","Calicut","Kochi","Thrissur","Calicut","Kochi","Thrissur"
] * 3,
    "churn": [0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,1,1,1,1,0,0,0,0,0,1,1,1,1,1]
}


df = pd.DataFrame(data)
print(df.head())
```

14. Expected Practical Questions with Complete Answers

Practical 1 - Create and Inspect the Dummy Dataset

Question: Create the customer DataFrame, display the first five rows, check its shape, column names, data types, and missing values.

```
df.head()
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
print(df.dtypes)
print(df.isnull().sum())
```

Expected interpretation: The dataset contains 30 customer rows. The numerical features can be used for clustering, while city is categorical. There should be no missing values in the provided dummy data.

Practical 2 - Create Useful Features

Question: Create estimated_monthly_value = visits_per_month × avg_order_value and loyalty_score = membership_months × visits_per_month.

```
df["estimated_monthly_value"] = df["visits_per_month"] * df["avg_order_value"]
df["loyalty_score"] = df["membership_months"] * df["visits_per_month"]
print(df[["customer_id", "estimated_monthly_value", "loyalty_score"]].head())
```

Explanation: These engineered features combine existing behavior into more directly useful business signals. The first estimates monthly purchase value; the second combines tenure with visit frequency.

Practical 3 - Encode the City Column

Question: Convert the city column into numerical features using One-Hot Encoding.

```
df_encoded = pd.get_dummies(df, columns=["city"], dtype=int)
print(df_encoded.filter(like="city_").head())
```

Expected result: New binary columns such as city_Calicut, city_Kochi, and city_Thrissur are created. This avoids imposing an artificial numeric order on city names.

Practical 4 - Standardize Numerical Features

Question: Standardize annual_income, spending_score, visits_per_month, and avg_order_value using StandardScaler.

```
from sklearn.preprocessing import StandardScaler
```

```
features = ["annual_income", "spending_score", "visits_per_month", "avg_order_value"]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[features])
print(X_scaled[:5])
```

Explanation: Scaling is important because annual income is measured in tens of thousands while visits and spending score use much smaller numeric ranges. Without scaling, income would dominate distance calculations.

Practical 5 - Find a Suitable K Using the Elbow Method

Question: Train K-Means models for K=1 to K=7, store inertia, and plot the Elbow graph.

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

inertias = []
for k in range(1, 8):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

plt.plot(range(1, 8), inertias, marker="o")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()
```

Expected interpretation: Look for the point at which the reduction in inertia begins to slow substantially. On this synthetic dataset, a small K such as 2 or 3 is likely to be reasonable, but the exact choice should be justified from the plotted curve.

Practical 6 - Build a K-Means Customer Segmentation Model

Question: Build K-Means with K=3 and add the cluster labels back to the DataFrame.

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df["cluster"] = kmeans.fit_predict(X_scaled)
print(df[["customer_id", "cluster"]].head(10))
```

Explanation: Each customer receives a cluster number. The numerical label itself has no business meaning until cluster statistics are analyzed.

Practical 7 - Interpret the Clusters

Question: Calculate the mean age, income, spending score, visits, and order value for each cluster.

```
summary = df.groupby("cluster")[[
    "age", "annual_income", "spending_score",
    "visits_per_month", "avg_order_value"
]].mean().round(2)

print(summary)
```

Expected interpretation: A cluster with lower income but high spending score and many visits could be labeled “high engagement”; a cluster with high income but low spending and few visits could be labeled “high income, low engagement”. The exact names must be based on the output rather than the cluster number.

Practical 8 - Calculate Silhouette Score

Question: Calculate the silhouette score for the K=3 clustering solution.

```
from sklearn.metrics import silhouette_score


score = silhouette_score(X_scaled, df["cluster"])
print("Silhouette Score:", round(score, 3))
```

Interpretation: A larger positive score indicates better-separated clusters. The score should be compared with other reasonable K values rather than treated as an isolated number.

Practical 9 - Perform Hierarchical Clustering

Question: Build an Agglomerative Clustering model with 3 clusters.

```
from sklearn.cluster import AgglomerativeClustering


hc = AgglomerativeClustering(n_clusters=3, linkage="ward")
df["hier_cluster"] = hc.fit_predict(X_scaled)
print(df[["customer_id", "hier_cluster"]].head(10))
```

Explanation: Hierarchical clustering forms a hierarchy of merges. Ward linkage attempts to merge clusters in a way that minimizes the increase in within-cluster variance.

Practical 10 - Draw a Dendrogram

Question: Draw a dendrogram for the standardized customer features.

```
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage, dendrogram


Z = linkage(X_scaled, method="ward")
plt.figure(figsize=(10, 5))
dendrogram(Z)
plt.title("Customer Dendrogram")
plt.xlabel("Customer Index")
plt.ylabel("Distance")
plt.show()
```

Expected interpretation: Observe large vertical gaps in the dendrogram. A horizontal cut through a large gap can suggest a suitable number of clusters.

Practical 11 - Apply PCA to Two Components

Question: Reduce the four standardized numerical features to two principal components and show explained variance.

```
from sklearn.decomposition import PCA


pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_scaled)
print("Explained variance ratio:", pca.explained_variance_ratio_)
print("Total explained variance:", pca.explained_variance_ratio_.sum())
```

Explanation: The two values show how much of the original variance is captured by PC1 and PC2. Their sum shows how much information remains after reducing the data to two dimensions.

Practical 12 - Visualize Clusters After PCA

Question: Plot PC1 versus PC2 and color the points by K-Means cluster.

```
plt.figure(figsize=(7, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=df["cluster"])
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Customer Clusters in PCA Space")
plt.show()
```

Expected interpretation: Points belonging to the same cluster should tend to appear near one another. Strong overlap suggests the groups may not be clearly separated in the selected feature space.

Practical 13 - Train a Classification Model for Churn

Question: Use age, annual_income, spending_score, visits_per_month, avg_order_value, and membership_months to train a Logistic Regression model that predicts churn.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

model_features = [
    "age", "annual_income", "spending_score",
    "visits_per_month", "avg_order_value", "membership_months"
]
X = df[model_features]
y = df["churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

Explanation: A Pipeline prevents inconsistent preprocessing by applying the scaler and model in sequence. Stratification helps preserve the class distribution in train and test sets.

Practical 14 - Calculate Accuracy, Precision, Recall, F1, and Confusion Matrix

Question: Evaluate the churn predictions using all metrics covered in the module.

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
print("F1:", f1_score(y_test, y_pred, zero_division=0))
print("Confusion Matrix:
", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, zero_division=0))
```

Expected interpretation: Do not report only Accuracy. Inspect Precision and Recall to understand false positives and false negatives, then use F1 when a balance is required.

Practical 15 - Perform 5-Fold Cross Validation

Question: Evaluate the Logistic Regression pipeline using 5-Fold Cross Validation.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(pipe, X, y, cv=5, scoring="f1")
print("Fold F1 scores:", scores)
print("Mean F1:", scores.mean())
print("Std F1:", scores.std())
```

Explanation: The mean estimates overall performance across folds. The standard deviation indicates consistency. Very different fold scores suggest instability.

Practical 16 - Grid Search for Decision Tree Hyperparameters

Question: Tune max_depth and min_samples_split of a DecisionTreeClassifier using GridSearchCV.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

params = {
    "max_depth": [2, 3, 4, None],
    "min_samples_split": [2, 4, 6]
}

grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
```

```
    param_grid=params,
    cv=5,
    scoring="f1"
)
grid.fit(X, y)
print("Best parameters:", grid.best_params_)
print("Best CV F1:", grid.best_score_)
```

Explanation: Grid Search evaluates every specified combination using cross validation and reports the best setting according to the chosen metric.

Practical 17 - Random Search for a Random Forest

Question: Use RandomizedSearchCV to tune a RandomForestClassifier.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    "n_estimators": [50, 100, 150, 200],
    "max_depth": [2, 3, 4, None],
    "min_samples_split": [2, 4, 6]
}

random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions=param_dist,
    n_iter=8,
    cv=5,
    scoring="f1",
    random_state=42
)
random_search.fit(X, y)
print(random_search.best_params_)
print(random_search.best_score_)
```

Explanation: Random Search evaluates only a sample of combinations, making it more efficient when the search space is larger.

Practical 18 - Compare Multiple Models

Question: Compare Logistic Regression, Decision Tree, and Random Forest using cross-validation F1 scores.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

models = {
    "Logistic Regression": Pipeline([
```

```
        ("scaler", StandardScaler()),
        ("model", LogisticRegression())
    ]),
    "Decision Tree": DecisionTreeClassifier(max_depth=3, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42)
}

for name, model in models.items():
    score = cross_val_score(model, X, y, cv=5, scoring="f1").mean()
    print(name, round(score, 3))
```

Expected interpretation: Select a model based on the relevant metric and consistency, not simply because one algorithm is more complex.

Practical 19 - Generate Actionable Business Insights

Question: From the customer cluster summary, write three business insights and corresponding actions.

Finding	Possible Interpretation	Possible Action
Cluster has high visits and high spending score	Strongly engaged customers	Offer loyalty rewards and early access
Cluster has high income but low visits	Untapped high-value potential	Run personalized re-engagement campaigns
Cluster shows low visits and declining engagement	Possible churn risk	Use retention offers or feedback surveys

Answer quality requirement: The recommendation must follow logically from the observed cluster statistics. Avoid unsupported statements such as claiming that a marketing action will definitely cause higher sales.

Practical 20 - Complete Mini Project

Question: Build a complete customer segmentation workflow using the dummy data and present the final findings.

43. Load the DataFrame.
44. Check missing values and duplicates.
45. Select income, spending score, visits, and average order value.
46. Standardize the features.
47. Use the Elbow Method and optionally silhouette score to choose K.
48. Fit K-Means.
49. Add cluster labels to the DataFrame.
50. Summarize cluster means.
51. Apply PCA to two components.
52. Plot clusters in PCA space.
53. Assign meaningful segment names.
54. Write at least three business recommendations.
55. State limitations of the small synthetic dataset.

Expected Exam Answer Pattern
For a full-mark practical, show the code, explain why each preprocessing step is needed, display or describe the result, and interpret the result in business language.

15. Additional Practice Tasks

1. Compare K=2, K=3, and K=4 using silhouette score and explain which is strongest.
2. Remove avg_order_value and rebuild K-Means. Explain how the clusters change.
3. Add estimated_monthly_value to the clustering features and compare the segment profiles.
4. Apply PCA with enough components to retain at least 90% variance.
5. Compare one-hot encoding and ordinal encoding for the city column and explain when each is appropriate.
6. Create a confusion matrix manually from a list of actual and predicted churn labels.
7. Calculate Accuracy, Precision, Recall, and F1 manually from TP, TN, FP, and FN.
8. Compare 3-fold and 5-fold cross-validation results.
9. Tune Decision Tree parameters using both Grid Search and Random Search and compare the number of combinations tested.
10. Write a one-page analytical story explaining the customer segments to a retail manager.

16. Quick Revision Sheet

Concept	One-Line Revision
Unsupervised Learning	Finds hidden structure in unlabeled data.
Clustering	Groups similar observations.
K-Means	Centroid-based clustering requiring K.
Hierarchical Clustering	Builds a hierarchy of clusters and can be visualized with a dendrogram.
PCA	Creates principal components that preserve maximum variance.
Feature Engineering	Creates or transforms useful model inputs.
Feature Selection	Keeps useful original variables.
Accuracy	Overall proportion of correct predictions.
Precision	How reliable positive predictions are.
Recall	How many actual positives are detected.
F1 Score	Balances Precision and Recall.
Cross Validation	Evaluates performance across multiple folds.
Hyperparameter	A model configuration value not directly learned as a fitted parameter.
Grid Search	Tests all specified parameter combinations.
Random Search	Tests randomly sampled parameter combinations.
Analytical Storytelling	Converts technical results into decision-friendly insights.

Final Student Checklist

- I can explain K-Means step by step.
- I know why scaling is required before distance-based clustering.
- I can interpret clusters rather than only print labels.
- I can explain PCA and explained variance.
- I can create and encode useful features.
- I can calculate and interpret classification metrics.
- I understand K-Fold Cross Validation.

- I know parameters vs hyperparameters.
- I can use Grid Search and Random Search.
- I can turn analytical output into clear business recommendations.